

# Concurrency in Node.js

---

## Introduction

---

Concurrency is the ability of a system to handle multiple tasks at the same time. In Node.js, concurrency is achieved through non-blocking I/O operations and the event loop, which allows the system to handle thousands of operations without creating separate threads for each task.

Think of concurrency like a chef in a kitchen. Instead of preparing one dish completely before starting another (sequential), the chef puts water to boil, while it's heating, they chop vegetables, then check on the boiling water, then prepare the sauce. This is concurrency—managing multiple tasks by switching between them efficiently.

However, Node.js has a single-threaded event loop, which means CPU-intensive tasks can block the entire application. This is where understanding the difference between blocking and non-blocking operations, and when to use worker threads, becomes crucial.

---

## Understanding Blocking vs Non-Blocking Operations

---

### What Does "Blocking" Mean?

Blocking operations halt the execution of further code until the operation completes. During this time, the entire Node.js process cannot handle other requests or execute other code. This is like standing at a door waiting for someone to unlock it—you can't do anything else until they do.

Non-blocking operations, on the other hand, allow the program to continue executing other code while waiting for the operation to complete. When the operation finishes, a callback is executed. This is like ringing a doorbell and continuing with other tasks until someone answers.

## The Critical Difference

```
// Example demonstrating the concept
console.log('Start of program');

// Imagine this is a blocking operation (synchronous)
console.log('Blocking operation started...');
// If this takes 5 seconds, nothing else can run
console.log('Blocking operation completed');

// Imagine this is a non-blocking operation (asynchronous)
console.log('Non-blocking operation started...');
setTimeout(() => {
  console.log('Non-blocking operation completed');
}, 5000);
// Program continues immediately without waiting

console.log('End of program');
```

### Output:

```
Start of program
Blocking operation started...
Blocking operation completed
Non-blocking operation started...
End of program
(after 5 seconds)
Non-blocking operation completed
```

**Key Observation:** With non-blocking operations, "End of program" executes immediately without waiting for the timeout to complete.

## File System Operations: Synchronous vs Asynchronous

### Understanding fs.readFileSync (Blocking)

The `fs.readFileSync()` method reads a file synchronously, blocking all code execution until the file is completely read. Let's see this in action.

#### Example 1: Basic Synchronous File Reading

```
const fs = require('fs');

console.log('1. Starting file read...');

try {
  // This blocks everything until file is read
  const data = fs.readFileSync('example.txt', 'utf8');
  console.log('2. File content:', data);
} catch (error) {
  console.error('2. Error reading file:', error.message);
}

console.log('3. This runs only after file is read');
console.log('4. Program continues...');
```

#### Output:

```
1. Starting file read...
2. File content: This is the content of example.txt file
3. This runs only after file is read
4. Program continues...
```

**Notice:** Every line executes in strict order. Line 3 and 4 must wait for the file reading to complete.

#### Example 2: The Blocking Problem with Multiple Requests

Let's simulate a scenario where multiple users request data from a server. With synchronous operations, each request must wait for the previous one to complete.

```
const fs = require('fs');

function handleRequest(userId) {
  console.log(`[${new Date().toISOString()}] Request from User ${userId} started`);

  // Simulate reading user data from file (blocking)
  try {
    const data = fs.readFileSync('user-data.txt', 'utf8');
    // Simulate some processing time
    const start = Date.now();
    while (Date.now() - start < 1000) {} // Block for 1 second

    console.log(`[${new Date().toISOString()}] Request from User ${userId} completed`);
  } catch (error) {
    console.error(`User ${userId} error:`, error.message);
  }
}

// Simulate 3 users making requests
console.log('Server started. Handling requests with SYNCHRONOUS operations...\n');
handleRequest(1);
handleRequest(2);
handleRequest(3);
console.log('\nAll requests processed');
```

#### Output:

```
Server started. Handling requests with SYNCHRONOUS operations...
```

```
[2026-02-16T10:00:00.000Z] Request from User 1 started
[2026-02-16T10:00:01.000Z] Request from User 1 completed
[2026-02-16T10:00:01.000Z] Request from User 2 started
[2026-02-16T10:00:02.000Z] Request from User 2 completed
[2026-02-16T10:00:02.000Z] Request from User 3 started
[2026-02-16T10:00:03.000Z] Request from User 3 completed
```

```
All requests processed
```

```
Total time: ~3 seconds
```

**Problem:** User 2 has to wait for User 1 to complete, and User 3 has to wait for both. The total time is the sum of all operations. In a real web server, this would mean all other users are completely blocked!

## Understanding fs.readFile (Non-Blocking)

The `fs.readFile()` method reads a file asynchronously, allowing other code to execute while the file is being read. This is the foundation of Node.js's scalability.

### Example 3: Basic Asynchronous File Reading

```
const fs = require('fs');

console.log('1. Starting file read...');

// Non-blocking file read
fs.readFile('example.txt', 'utf8', (error, data) => {
  if (error) {
    console.error('Error reading file:', error.message);
    return;
  }
  console.log('3. File content:', data);
});

console.log('2. This runs immediately, without waiting!');
console.log('4. Program continues executing...');
```

#### Output:

```
1. Starting file read...
2. This runs immediately, without waiting!
4. Program continues executing...
(after file read completes)
3. File content: This is the content of example.txt file
```

**Notice:** Lines 2 and 4 execute immediately without waiting for the file to be read. The file content appears later, when the operation completes.

#### Example 4: Multiple Requests with Non-Blocking I/O

Now let's see the same scenario with asynchronous operations. The difference is dramatic.

```
const fs = require('fs');

function handleRequestAsync(userId) {
  console.log(`[${new Date().toISOString()}] Request from User ${userId} started`);

  // Non-blocking file read
  fs.readFile('user-data.txt', 'utf8', (error, data) => {
    if (error) {
      console.error(`User ${userId} error:`, error.message);
      return;
    }

    // Simulate processing time (non-blocking)
    setTimeout(() => {
      console.log(`[${new Date().toISOString()}] Request from User ${userId} completed`);
    }, 1000);
  });
}

// Simulate 3 users making requests
console.log('Server started. Handling requests with ASYNCHRONOUS operations...\n');
handleRequestAsync(1);
handleRequestAsync(2);
handleRequestAsync(3);
console.log('\nAll requests initiated (not completed yet)');
```

#### Output:

```
Server started. Handling requests with ASYNCHRONOUS operations...

[2026-02-16T10:00:00.000Z] Request from User 1 started
[2026-02-16T10:00:00.000Z] Request from User 2 started
[2026-02-16T10:00:00.000Z] Request from User 3 started

All requests initiated (not completed yet)
(after ~1 second, all complete nearly simultaneously)
[2026-02-16T10:00:01.000Z] Request from User 1 completed
[2026-02-16T10:00:01.000Z] Request from User 2 completed
```

[2026-02-16T10:00:01.000Z] Request from User 3 completed

Total time: ~1 second

**Huge Improvement:** All three requests are processed concurrently! The total time is approximately 1 second instead of 3 seconds. No user has to wait for others to complete.

## Side-by-Side Comparison

```
const fs = require('fs');

// Demonstrating both approaches with timing
function demonstrateBlocking() {
  console.log('=== BLOCKING APPROACH ===');
  const startTime = Date.now();

  try {
    const file1 = fs.readFileSync('file1.txt', 'utf8');
    const file2 = fs.readFileSync('file2.txt', 'utf8');
    const file3 = fs.readFileSync('file3.txt', 'utf8');

    const totalTime = Date.now() - startTime;
    console.log(`Files read: ${file1.length + file2.length + file3.length} bytes`);
    console.log(`Time taken: ${totalTime}ms`);
    console.log(`Approach: Sequential - one after another\n`);
  } catch (error) {
    console.error('Error:', error.message);
  }
}

function demonstrateNonBlocking() {
  console.log('=== NON-BLOCKING APPROACH ===');
  const startTime = Date.now();
  let filesRead = 0;
  let totalBytes = 0;

  // Read all files concurrently
  fs.readFile('file1.txt', 'utf8', (err, data) => {
    if (!err) {
      filesRead++;
      totalBytes += data.length;
      checkComplete();
    }
  })
}
```

```
});

fs.readFile('file2.txt', 'utf8', (err, data) => {
  if (!err) {
    filesRead++;
    totalBytes += data.length;
    checkComplete();
  }
});

fs.readFile('file3.txt', 'utf8', (err, data) => {
  if (!err) {
    filesRead++;
    totalBytes += data.length;
    checkComplete();
  }
});

function checkComplete() {
  if (filesRead === 3) {
    const totalTime = Date.now() - startTime;
    console.log(`Files read: ${totalBytes} bytes`);
    console.log(`Time taken: ${totalTime}ms`);
    console.log(`Approach: Concurrent - all at once\n`);
  }
}

// Run demonstrations
demonstrateBlocking();
setTimeout(() => demonstrateNonBlocking(), 100);
```

#### Output:

```
=== BLOCKING APPROACH ===
Files read: 15000 bytes
Time taken: 45ms
Approach: Sequential - one after another

=== NON-BLOCKING APPROACH ===
Files read: 15000 bytes
```



Time taken: 18ms

Approach: Concurrent - all at once

**Performance Gain:** Non-blocking I/O is typically 2-3x faster for parallel operations and doesn't block other operations in your application.

## When Blocking Becomes Critical

### Real-World Server Example

Let's simulate a real HTTP server scenario where the choice between blocking and non-blocking becomes critical.

```
const http = require('http');
const fs = require('fs');

// Server with BLOCKING operations
const blockingServer = http.createServer((req, res) => {
  console.log(`[Blocking] Request received at ${new Date().toISOString()}`);

  // This blocks the entire server!
  try {
    const data = fs.readFileSync('large-file.txt', 'utf8');
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end(`Data length: ${data.length} characters`);
  } catch (error) {
    res.writeHead(500);
    res.end('Error reading file');
  }
});

// Server with NON-BLOCKING operations
const nonBlockingServer = http.createServer((req, res) => {
  console.log(`[Non-blocking] Request received at ${new Date().toISOString()}`);

  // This doesn't block other requests
  fs.readFile('large-file.txt', 'utf8', (error, data) => {
    if (error) {
      res.writeHead(500);
    }
  });
});
```

```
        res.end('Error reading file');
        return;
    }

    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end(`Data length: ${data.length} characters`);
  });
});

// Start servers on different ports
blockingServer.listen(3000, () => {
  console.log('Blocking server running on port 3000');
});

nonBlockingServer.listen(3001, () => {
  console.log('Non-blocking server running on port 3001');
});
```

#### Output:

Blocking server running on port 3000

Non-blocking server running on port 3001

Scenario: 100 simultaneous requests

Blocking server: Handles 1 request at a time, others wait

- Request 1: 50ms
- Request 2: Must wait for #1, starts at 50ms
- Request 100: Starts at 4950ms

Total time: ~5000ms (5 seconds)

Non-blocking server: Handles all requests concurrently

- All 100 requests start immediately
- All complete in ~200ms

Total time: ~200ms

**Impact:** In a blocking scenario, the 100th user waits 5 seconds! In a non-blocking scenario, all users get responses in under a second. This is why Node.js excels at I/O-heavy applications.

## The Golden Rule of Node.js

Never use synchronous (blocking) operations in production server code.

Use synchronous operations only for:

- Command-line scripts and utilities
- Application initialization (loading config files at startup)
- Development and testing scenarios

---

## CPU-Bound vs I/O-Bound Operations

### Understanding the Difference

Before we discuss worker threads, we need to understand two types of operations:

- **I/O-Bound Operations:** Operations waiting for external resources (file system, database, network). These benefit from asynchronous I/O.
- **CPU-Bound Operations:** Operations requiring heavy computation (image processing, encryption, complex calculations). These block the event loop even if written asynchronously.

### Example 5: CPU-Bound Operation Problem

```
const http = require('http');

// CPU-intensive function
function calculatePrimes(max) {
  const primes = [];
  for (let i = 2; i <= max; i++) {
    let isPrime = true;
    for (let j = 2; j <= Math.sqrt(i); j++) {
      if (i % j === 0) {
        isPrime = false;
        break;
      }
    }
    if (isPrime) primes.push(i);
  }
}
```

```
    return primes;
  }

  const server = http.createServer((req, res) => {
    if (req.url === '/primes') {
      console.log(`[${new Date().toISOString()}] Prime calculation request started`);

      // This is CPU-intensive and WILL block the event loop
      const primes = calculatePrimes(100000);

      res.writeHead(200, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ count: primes.length, sample: primes.slice(0, 10) }));

      console.log(`[${new Date().toISOString()}] Prime calculation completed`);
    } else {
      res.writeHead(200, { 'Content-Type': 'text/plain' });
      res.end('Hello World');
    }
  });

  server.listen(3000, () => {
    console.log('Server running on port 3000');
    console.log('Try accessing /primes and / simultaneously');
  });
```

#### Output:

```
Server running on port 3000
Try accessing /primes and / simultaneously

User 1 requests /primes
[2026-02-16T10:00:00.000Z] Prime calculation request started
(server is blocked for 2 seconds during calculation)
[2026-02-16T10:00:02.000Z] Prime calculation completed

User 2 requests / (simple response)
(had to wait 2 seconds because event loop was blocked)
Response received after 2 seconds
```

**Problem:** Even though `/` is a simple request, it has to wait for the prime calculation to finish. The CPU-intensive task blocks ALL other operations, even I/O operations!

---

## Worker Threads: Solution for CPU-Intensive Tasks

---

### What are Worker Threads?

Worker threads allow you to execute JavaScript code in parallel on separate threads, preventing CPU-intensive operations from blocking the main event loop. Think of them as hiring additional workers to handle heavy lifting, while the main worker (event loop) continues greeting customers and handling quick tasks.

Worker threads were introduced in Node.js v10.5.0 and became stable in v12.0.0. They provide true parallel execution for JavaScript code.

### When to Use Worker Threads

| Use Worker Threads For                                                                                                                                                                                                                                                              | Don't Use Worker Threads For                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Heavy calculations (cryptography, data processing)</li><li>• Image/video processing</li><li>• Large data transformations</li><li>• Complex algorithms (sorting, searching large datasets)</li><li>• Machine learning computations</li></ul> | <ul style="list-style-type: none"><li>• Simple I/O operations (file reads, database queries)</li><li>• Network requests</li><li>• Simple calculations</li><li>• Operations already non-blocking</li><li>• Frequently triggered short tasks</li></ul> |

### Worker Thread Basics

#### Example 6: Basic Worker Thread Setup

Worker threads consist of two parts: the main thread and the worker thread. Let's create a simple example.

```
// main.js - Main thread
const { Worker } = require('worker_threads');

console.log('Main thread: Starting worker...');

// Create a new worker thread
const worker = new Worker('./worker.js');

// Listen for messages from worker
worker.on('message', (result) => {
  console.log('Main thread: Received result from worker:', result);
});

// Listen for errors
worker.on('error', (error) => {
  console.error('Main thread: Worker error:', error);
});

// Listen for worker exit
worker.on('exit', (code) => {
  if (code !== 0) {
    console.error(`Main thread: Worker stopped with exit code ${code}`);
  } else {
    console.log('Main thread: Worker completed successfully');
  }
});

console.log('Main thread: Continuing with other work...');
console.log('Main thread: I am not blocked!');
```

```
// worker.js - Worker thread
const { parentPort } = require('worker_threads');

console.log('Worker thread: Starting calculation...');

// Simulate CPU-intensive work
let sum = 0;
for (let i = 0; i < 1000000000; i++) {
  sum += i;
}

console.log('Worker thread: Calculation complete');
```

```
// Send result back to main thread
parentPort.postMessage({ sum, message: 'Calculation finished!' });
```

#### Output:

```
Main thread: Starting worker...
Main thread: Continuing with other work...
Main thread: I am not blocked!
Worker thread: Starting calculation...
Worker thread: Calculation complete
Main thread: Received result from worker: { sum: 499999999500000000, message: 'Calculation finished!' }
Main thread: Worker completed successfully
```

**Key Observation:** The main thread continues executing immediately while the worker performs the calculation in parallel. The main thread is never blocked!

#### Example 7: Passing Data to Workers

```
// main-with-data.js
const { Worker } = require('worker_threads');

function runWorker(workerData) {
  return new Promise((resolve, reject) => {
    // Pass data to worker via workerData option
    const worker = new Worker('./calculator-worker.js', {
      workerData: workerData
    });

    worker.on('message', resolve);
    worker.on('error', reject);
    worker.on('exit', (code) => {
      if (code !== 0) {
        reject(new Error(`Worker stopped with exit code ${code}`));
      }
    });
  });
}

// Run multiple calculations in parallel
```

```

async function runCalculations() {
  console.log('Starting multiple calculations in parallel...\n');

  const calculations = [
    { operation: 'factorial', number: 20 },
    { operation: 'fibonacci', number: 40 },
    { operation: 'primes', max: 100000 }
  ];

  const startTime = Date.now();

  // Run all workers in parallel
  const results = await Promise.all(
    calculations.map(calc => runWorker(calc))
  );

  const endTime = Date.now();

  console.log('\nAll calculations completed!');
  results.forEach((result, index) => {
    console.log(`Result ${index + 1}:`, result);
  });
  console.log(`\nTotal time: ${endTime - startTime}ms`);
}

runCalculations();

```

```

// calculator-worker.js
const { parentPort, workerData } = require('worker_threads');

function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}

function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

function countPrimes(max) {
  let count = 0;
  for (let i = 2; i <= max; i++) {
    let isPrime = true;
    for (let j = 2; j <= Math.sqrt(i); j++) {

```



```
        if (i % j === 0) {
            isPrime = false;
            break;
        }
    }
    if (isPrime) count++;
}
return count;
}

// Perform calculation based on workerData
const { operation, number, max } = workerData;
let result;

switch (operation) {
    case 'factorial':
        result = factorial(number);
        break;
    case 'fibonacci':
        result = fibonacci(number);
        break;
    case 'primes':
        result = countPrimes(max);
        break;
    default:
        throw new Error('Unknown operation');
}

parentPort.postMessage({
    operation,
    input: number || max,
    result
});
```

#### Output:

```
Starting multiple calculations in parallel...
```

```
All calculations completed!
```

```
Result 1: { operation: 'factorial', input: 20, result: 2432902008176640000 }
```

```
Result 2: { operation: 'fibonacci', input: 40, result: 102334155 }
```

```
Result 3: { operation: 'primes', input: 100000, result: 9592 }
```

Total time: 892ms

(Without worker threads, running sequentially: ~2500ms)

**Performance Benefit:** All three calculations run in parallel, completing in roughly the time of the slowest operation rather than the sum of all operations.

### Example 8: Solving the Prime Number Server Problem

Let's fix our earlier CPU-intensive server using worker threads.

```
// server-with-workers.js
const http = require('http');
const { Worker } = require('worker_threads');

function calculatePrimesInWorker(max) {
  return new Promise((resolve, reject) => {
    const worker = new Worker(`
      const { parentPort, workerData } = require('worker_threads');

      function calculatePrimes(max) {
        const primes = [];
        for (let i = 2; i <= max; i++) {
          let isPrime = true;
          for (let j = 2; j <= Math.sqrt(i); j++) {
            if (i % j === 0) {
              isPrime = false;
              break;
            }
          }
          if (isPrime) primes.push(i);
        }
        return primes;
      }

      const primes = calculatePrimes(workerData);
      parentPort.postMessage(primes);
    `, {
      eval: true,
      workerData: max
    });
  });
}
```

```
    worker.on('message', resolve);
    worker.on('error', reject);
  });
}

const server = http.createServer(async (req, res) => {
  if (req.url === '/primes') {
    console.log(`[${new Date().toISOString()}] Prime calculation request - using worker`);

    try {
      // Calculation happens in worker thread
      const primes = await calculatePrimesInWorker(100000);

      res.writeHead(200, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({
        count: primes.length,
        sample: primes.slice(0, 10)
      }));

      console.log(`[${new Date().toISOString()}] Prime calculation completed`);
    } catch (error) {
      res.writeHead(500);
      res.end('Error calculating primes');
    }
  } else {
    // This responds immediately even if /primes is calculating
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Hello World - I respond instantly!');
  }
});

server.listen(3000, () => {
  console.log('Server running on port 3000 with worker threads');
  console.log('Main event loop remains responsive!');
});
```

#### Output:

```
Server running on port 3000 with worker threads
Main event loop remains responsive!

User 1 requests /primes
[2026-02-16T10:00:00.000Z] Prime calculation request - using worker
```

```
User 2 requests / (immediately after User 1)
Response: "Hello World - I respond instantly!" (received immediately!)

(2 seconds later)
[2026-02-16T10:00:02.000Z] Prime calculation completed
User 1 receives prime numbers
```

**Fixed!** User 2 gets an instant response even though User 1's heavy calculation is still running. The main thread remains responsive.

## Worker Thread Pool Pattern

For applications that need to handle many CPU-intensive tasks, creating a worker pool is more efficient than creating a new worker for each task.

```
// worker-pool.js
const { Worker } = require('worker_threads');

class WorkerPool {
  constructor(workerScript, poolSize = 4) {
    this.workerScript = workerScript;
    this.poolSize = poolSize;
    this.workers = [];
    this.queue = [];

    // Create worker pool
    for (let i = 0; i < poolSize; i++) {
      this.workers.push({
        worker: null,
        busy: false,
        id: i
      });
    }

    console.log(`Worker pool created with ${poolSize} workers`);
  }

  async execute(data) {
    return new Promise((resolve, reject) => {
      // Add task to queue
      this.queue.push({ data, resolve, reject });
      this.processQueue();
    });
  }
}
```

```
});  
}  
  
processQueue() {  
  if (this.queue.length === 0) return;  
  
  // Find available worker  
  const availableWorker = this.workers.find(w => !w.busy);  
  
  if (!availableWorker) {  
    console.log('All workers busy. Task queued.');    return;  
  }  
  
  // Get next task  
  const { data, resolve, reject } = this.queue.shift();  
  
  // Mark worker as busy  
  availableWorker.busy = true;  
  
  console.log(`Worker ${availableWorker.id} starting task`);  
  
  // Create worker  
  const worker = new Worker(this.workerScript, { workerData: data });  
  availableWorker.worker = worker;  
  
  worker.on('message', (result) => {  
    console.log(`Worker ${availableWorker.id} completed task`);  
    availableWorker.busy = false;  
    availableWorker.worker = null;  
    resolve(result);  
  
    // Process next task in queue  
    this.processQueue();  
  });  
  
  worker.on('error', (error) => {  
    console.error(`Worker ${availableWorker.id} error:`, error);  
    availableWorker.busy = false;  
    availableWorker.worker = null;  
    reject(error);  
  
    // Process next task in queue  
    this.processQueue();  
  });  
}  
}
```

```
// Usage
async function demonstratePool() {
  const pool = new WorkerPool('./calculator-worker.js', 2);

  console.log('\nSubmitting 5 tasks to pool with 2 workers...\n');

  const tasks = [
    { operation: 'fibonacci', number: 35 },
    { operation: 'fibonacci', number: 36 },
    { operation: 'fibonacci', number: 37 },
    { operation: 'fibonacci', number: 38 },
    { operation: 'fibonacci', number: 39 }
  ];

  const startTime = Date.now();
  const results = await Promise.all(tasks.map(task => pool.execute(task)));
  const endTime = Date.now();

  console.log('\nAll tasks completed!');
  console.log('Results:', results);
  console.log(`Total time: ${endTime - startTime}ms`);
}

demonstratePool();
```

#### Output:

```
Worker pool created with 2 workers

Submitting 5 tasks to pool with 2 workers...

Worker 0 starting task
Worker 1 starting task
All workers busy. Task queued.
All workers busy. Task queued.
All workers busy. Task queued.
Worker 0 completed task
Worker 0 starting task
Worker 1 completed task
Worker 1 starting task
Worker 0 completed task
Worker 1 completed task
```

All tasks completed!

Results: [

```
{ operation: 'fibonacci', input: 35, result: 9227465 },  
{ operation: 'fibonacci', input: 36, result: 14930352 },  
{ operation: 'fibonacci', input: 37, result: 24157817 },  
{ operation: 'fibonacci', input: 38, result: 39088169 },  
{ operation: 'fibonacci', input: 39, result: 63245986 }
```

]

Total time: 3421ms

(Without pool, creating 5 workers: might run out of memory or be slower due to worker creation overhead)

### Benefits of Worker Pool:

- Reuses workers instead of creating new ones for each task
- Limits concurrent CPU usage by limiting pool size
- Automatically queues tasks when all workers are busy
- More memory efficient for handling many tasks

## Best Practices for Concurrency in Node.js

### 1. Choose the Right Tool

```
// ☒ Good: Async I/O for file operations  
fs.readFile('data.txt', 'utf8', (err, data) => {  
  // Handle data  
});  
  
// ☒ Bad: Sync I/O in server code  
const data = fs.readFileSync('data.txt', 'utf8'); // Blocks everything!  
  
// ☒ Good: Worker threads for CPU-intensive tasks  
const worker = new Worker('./heavy-computation.js');  
  
// ☒ Bad: CPU-intensive task in main thread
```

```
function complexCalculation() {  
  // Long calculation blocks event loop  
}
```

## 2. Don't Overuse Worker Threads

```
// ✗ Bad: Using worker for simple task  
const worker = new Worker(`  
  const { parentPort, workerData } = require('worker_threads');  
  parentPort.postMessage(workerData.a + workerData.b);  
`, { eval: true, workerData: { a: 5, b: 3 } });  
  
// ☑ Good: Simple tasks in main thread  
const result = 5 + 3; // No overhead!
```

## 3. Proper Error Handling

```
// ☑ Good: Handle all worker events  
const worker = new Worker('./worker.js');  
  
worker.on('message', (result) => {  
  console.log('Success:', result);  
});  
  
worker.on('error', (error) => {  
  console.error('Worker error:', error);  
});  
  
worker.on('exit', (code) => {  
  if (code !== 0) {  
    console.error('Worker exited with error');  
  }  
});
```

## 4. Clean Up Workers

```
// ☑ Good: Terminate workers when done  
async function processData(data) {  
  const worker = new Worker('./processor.js', { workerData: data });
```



```

try {
  const result = await new Promise((resolve, reject) => {
    worker.on('message', resolve);
    worker.on('error', reject);
  });

  return result;
} finally {
  await worker.terminate(); // Clean up
}
}

```

## Performance Comparison Summary

| Scenario            | Blocking (Sync)                        | Non-Blocking (Async I/O)                        | Worker Threads                            |
|---------------------|----------------------------------------|-------------------------------------------------|-------------------------------------------|
| File I/O            | ✗ Blocks everything<br>~50ms per file  | ☑ Concurrent<br>~50ms total for multiple        | ⚠ Unnecessary overhead                    |
| Network Requests    | ✗ Blocks everything<br>Sequential only | ☑ Highly efficient<br>Thousands concurrent      | ⚠ Unnecessary overhead                    |
| CPU-Intensive       | ✗ Blocks event loop<br>Single-threaded | ✗ Still blocks event loop<br>Async doesn't help | ☑ True parallelism<br>Uses multiple cores |
| Simple Calculations | ☑ Fast and simple<br>No overhead       | ☑ Fast and simple<br>No overhead                | ✗ Too much overhead<br>Slower than direct |

## Real-World Decision Flow

```
Is your operation I/O-bound (file, network, database)?
├─ YES → Use async/await with non-blocking I/O
│   └─ ☒ fs.readFile, fetch, http requests, db queries
└─ NO → Is it CPU-intensive?
    └─ YES → How long does it take?
        ├── < 10ms → Do it in main thread
        │   └─ ☒ Simple math, string operations
        └─ > 10ms → Use worker threads
            └─ ☒ Image processing, complex algorithms,
                heavy calculations, data transformation
    └─ NO → Is it a simple synchronous operation?
        └─ YES → Do it directly in main thread
            └─ ☒ Simple calculations, data structures
```

## Interview Questions

### Theoretical Questions

**1. What is the difference between blocking and non-blocking I/O in Node.js?**

*Answer: Blocking I/O (synchronous) halts the execution of further code until the operation completes, making the entire application wait. Non-blocking I/O (asynchronous) allows the program to continue executing other code while waiting for the operation to complete. In Node.js, blocking I/O can make the entire server unresponsive as it has a single-threaded event loop. Non-blocking I/O is the foundation of Node.js scalability, allowing it to handle thousands of concurrent operations.*

**2. Why should you never use `fs.readFileSync` in production server code?**

*Answer: `fs.readFileSync` blocks the entire event loop until the file is completely read, preventing the server from handling any other requests during that time. If the file is large or the disk is slow, all users must wait. In a production environment with multiple concurrent users, this can make the application appear frozen. The only acceptable use of sync methods is during application startup (loading config files) or in command-line tools where blocking is expected behavior.*

**3. What is the difference between CPU-bound and I/O-bound operations? Give examples.**

*Answer: I/O-bound operations spend most of their time waiting for input/output resources (file system, network, database) and benefit from async I/O. Examples: reading files, database queries, API calls. CPU-bound operations require heavy computational processing and are limited by CPU speed. Examples: image processing, encryption, complex calculations, video encoding. I/O-bound operations should use async/await, while CPU-bound operations need worker threads to avoid blocking the event loop.*

**4. What are Worker Threads in Node.js and when should you use them?**

*Answer: Worker threads enable true parallel execution of JavaScript code on separate threads, preventing CPU-intensive operations from blocking the main event loop. They should be used for heavy computational tasks like complex algorithms, image/video processing, cryptography, and large data transformations. They should NOT be used for I/O operations (which are already non-blocking) or simple calculations (where the overhead of creating a thread exceeds the benefit). A good rule: use workers for tasks taking more than 10ms of CPU time.*

**5. How does Node.js handle concurrent requests with a single-threaded event loop?**

*Answer: Node.js uses non-blocking I/O and the event loop. When an I/O operation is initiated (like reading a file), Node.js delegates it to the system's async I/O facilities and continues processing other requests. When the I/O completes, a callback is queued in the event loop. This allows Node.js to handle thousands of concurrent connections without creating a thread per connection. However, CPU-intensive tasks still block the event loop, which is why worker threads are needed for such operations.*

**6. What happens when you perform a CPU-intensive operation in the main thread?**

*Answer: A CPU-intensive operation in the main thread blocks the entire event loop, preventing Node.js from processing any other requests or callbacks. All incoming HTTP requests, timers, and I/O callbacks must wait until the CPU-intensive task completes. This makes the application appear frozen to all users. Even simple requests that should respond instantly will be delayed. This is why CPU-intensive tasks must be offloaded to worker threads.*

**7. Explain the concept of a worker pool. Why is it better than creating a new worker for each task?**

*Answer: A worker pool maintains a fixed number of reusable worker threads. Instead of creating and destroying workers for each task, tasks are queued and assigned to available workers. Benefits: (1) Avoids overhead of creating/destroying threads repeatedly, (2) Controls resource usage by limiting concurrent workers, (3) More efficient memory usage, (4) Better performance for handling many small tasks, (5) Prevents system overload from too many threads. Creating a new worker per task is only efficient for long-running, infrequent operations.*

**8. Can async/await help with CPU-intensive operations? Why or why not?**

*Answer: No, async/await cannot help with CPU-intensive operations. Async/await is designed for I/O-bound operations where the program waits for external resources. A CPU-intensive calculation, even when wrapped in a Promise or async function, still runs in the main thread and blocks the event loop. Async/await just changes the syntax; it doesn't create parallelism. For CPU-intensive tasks, you must use worker threads to achieve true parallel execution on separate CPU cores.*

**9. What are the performance implications of using synchronous vs asynchronous file operations?**

*Answer: Synchronous operations process files one at a time sequentially, with total time equal to the sum of all operations. Asynchronous operations can process multiple files concurrently, with total time approximately equal to the slowest operation. In a server context, synchronous operations make the entire application unresponsive to all users during file I/O, while asynchronous operations allow the server to handle other requests. Async operations typically provide 2-3x better performance and unlimited scalability compared to sync operations.*

**10. What is the overhead of creating a worker thread? When is this overhead justified?**

*Answer: Creating a worker thread has overhead including: (1) thread creation time (~50-100ms), (2) memory allocation for thread stack, (3) data serialization when passing to worker, (4) context switching costs. This overhead is justified when: (1) the task takes significantly longer than thread creation time (>100ms), (2) the task is CPU-intensive and would block the event loop, (3) you're using a worker pool where threads are reused. The overhead is NOT justified for simple operations taking <10ms or for I/O operations that are already non-blocking.*

## Practical Questions for Students to Solve

**1. Question 1: Convert Sync to Async**

The following code uses synchronous file operations. Convert it to asynchronous using promises or async/await:

```
function processFiles() {  
  const file1 = fs.readFileSync('input1.txt', 'utf8');  
  const file2 = fs.readFileSync('input2.txt', 'utf8');  
  const combined = file1 + file2;  
  fs.writeFileSync('output.txt', combined);  
  console.log('Processing complete');  
}
```

```
}  
  
processFiles();
```

## 2. Question 2: Identify Blocking Code

Review this HTTP server code and identify all blocking operations. Explain why each is problematic and suggest fixes:

```
const http = require('http');  
const fs = require('fs');  
  
const server = http.createServer((req, res) => {  
  const config = JSON.parse(fs.readFileSync('config.json', 'utf8'));  
  
  let result = 0;  
  for (let i = 0; i < 1000000000; i++) {  
    result += Math.sqrt(i);  
  }  
  
  const log = `Request at ${new Date()}\n`;  
  fs.writeFileSync('log.txt', log, { flag: 'a' });  
  
  res.end(`Result: ${result}`);  
});  
  
server.listen(3000);
```

## 3. Question 3: Implement Worker Thread for Image Processing

Create a simple image processing system using worker threads. Requirements:

- Main thread: HTTP server that accepts image upload requests
- Worker thread: Simulates image processing (use a loop to simulate CPU work)
- Main thread should remain responsive during processing
- Return processing status and result

## 4. Question 4: Parallel File Processing

Write an async function that:

- Reads 10 files concurrently (not sequentially)
- Processes each file's content (count words)

- Combines results into a single summary
- Handles errors for individual files without failing entire operation
- Measure and log the time difference between sequential and parallel approaches

#### 5. Question 5: Build a Task Queue with Worker Pool

Implement a TaskQueue class that:

- Maintains a pool of 3 worker threads
- Accepts tasks and queues them if all workers are busy
- Each task is a CPU-intensive operation (e.g., calculating fibonacci numbers)
- Reports when tasks start, complete, and when queue is empty
- Includes proper error handling and worker cleanup

---

## Summary

---

Concurrency in Node.js is achieved through two complementary mechanisms: non-blocking I/O for I/O-bound operations and worker threads for CPU-bound operations. Understanding when and how to use each is crucial for building high-performance Node.js applications.

### Key Takeaways:

- Non-blocking I/O is the foundation of Node.js scalability and efficiency
- Never use synchronous operations (`fs.readFileSync`) in production server code
- Asynchronous operations allow Node.js to handle thousands of concurrent requests
- CPU-intensive tasks block the event loop even when written asynchronously
- Worker threads provide true parallelism for CPU-bound operations
- Use worker threads only for tasks that justify the overhead (typically > 100ms of CPU time)
- Worker pools are more efficient than creating workers for each task
- I/O-bound operations: Use `async/await` and non-blocking APIs
- CPU-bound operations: Use worker threads to prevent blocking
- Simple operations: Execute directly in the main thread
- Always handle errors in both async operations and worker threads
- Clean up worker threads after use to prevent memory leaks



Mastering concurrency patterns in Node.js enables you to build scalable, high-performance applications that can handle high loads while remaining responsive. The key is choosing the right tool for each type of operation.

